OPEN-SOURCE SOFTWARE

Title: Open-Source Audit – Python

Name: Suyash Srivastava

Course: Open-source software

Institution: VIT Bhopal University

Platform: VITyarthi

Date of submission: 27/03/2026

PROJECT REPORT

PART A – ORIGIN AND PHILOSOPHY:

A1:

In the late 1980s, programming was powerful but inefficient for everyday developers. Languages like C and C++ offered high performance but required complex syntax and deep technical understanding, making them difficult to use for rapid development. On the other hand, existing script languages lacked structure and readability, often becoming hard to maintain as projects grew.

It was in this context that Guido Van Rossum, a Dutch programmer working at Central Wiskunde and Informatica decided to create a new programming language in 1989. His goal was not just to build another tool, but to solve a practical frustration, i.e., the lack of a language that balanced simplicity, readability, and power.

Python was designed with a strong emphasis on code readability and developer productivity. Unlike many existing languages, it used clean and minimal syntax, allowing programmers to express ideas in fewer lines of code. This made it especially attractive for beginners while still being powerful enough for experienced developers.

A defining aspect of Python's origin is that it was released as open-source software from the very beginning. Instead of keeping control over the language, Guido chose to make it freely available so that others could use, modify and improve it. This decision aligned with the broader philosophy of open-source, where collaboration and transparency are prioritized over ownership and restriction.

Over time, this open approach allowed Python to evolve beyond what a single developer or organization could achieve. Contributors from a global community helped expand its capabilities, leading to its adoption in diverse fields such as web development, data science, artificial intelligence, and automation.

The origin of Python therefore reflects not just a technical solution, but a philosophical one, i.e., a belief that software should be accessible, understandable,

and collectively improved. It was created to remove barriers in programming, and its open-source nature ensured that those barriers would continue to stay low for future generations of developers.

A2:

Python is released under the Python Software Foundation License (PSFL). It is a permissive open-source license. It is like MIT and BSD.

The Four Freedoms:

1. Freedom to run the program for any purpose.
2. Freedom to study how it works.
3. Freedom to modify it.
4. Freedom to distribute copies (original or modified).

Python offers all four freedoms.

Since PSF is a non-copyleft (permissive) license, companies can modify Python, they can sell products built on it. Companies do not have to release their modifications publicity. This means, python encourages adoption and business use and prioritizes growth over control.

Difference between GPL and MIT/PSF:

If you have a GPL license for any software you built, if you make any changes or modify it, you MUST open-source it. GPL license forces sharing as it believes on "Freedom must be preserved" philosophy.

Meanwhile, if you have a MIT/PSF license, if you modify your software, you are allowed to make the changes you made private to yourself. It has minimal restrictions as its philosophy is "Freedom includes the choice not to share."

GPL protects the ecosystem from privatization, while PSF encourages wider adoption by reducing restrictions.

FREE BEER vs FREE AS FREEDOM

Free Bear: No cost. For eg.: Free download

Free as Freedom: You control the software. You can modify and redistribute it.

Python allows developers to not only use the language at no cost, but also inspect its source code, modify its behavior, and redistribute it, which reflects "freedom" rather than just "zero price."

A3:

The idea that all software should be open source is appealing, but not entirely practical. Open source promotes transparency, collaboration, and faster innovation, as developers can study and improve existing code. It also makes technology more accessible. However, not all software can follow this model. Many companies depend on proprietary systems for revenue and competitive advantage, and making everything open source could weaken their business sustainability. Therefore, while open source is highly valuable for shared infrastructure and learning, it may may not be suitable for every type of software.

The concept of "standing on the shoulders of giants" highlights how modern software builds on existing technologies. Open source makes this possible by allowing developers to reuse and improve prior work.

PART B – LINUX FOOTPRINT (PYTHON)

1. Installation Method:
   Python was installed on the Linux system using the Advanced Package Tool (API), which is the default package manager in Ubuntu-based distributions. The installation process involved updating the package list using "sudo apt update" followed by installing Python with "sudo apt install python3". This method provides a precompiled and stable version of Python directly from the official repositories, eliminating the need for manual compilation.

```
suyas@LAPTOP-4AKMHPB5:~$ python3 --version
Python 3.12.3
```

```
suyas@LAPTOP-4AKMHPB5:~$ which python3
/usr/bin/python3
suyas@LAPTOP-4AKMHPB5:~$ python3 -c "import sys; print(sys.path
> )"
['', '/usr/lib/python312.zip', '/usr/lib/python3.12', '/usr/lib/python3.12/l
ib-dynload', '/usr/local/lib/python3.12/dist-packages', '/usr/lib/python3/di
st-packages']
```

2. <u>Key Directories:</u>

   The location of the Python executable was identified using the **which python3** command, which showed that the binary is stored in **/usr/bin/python3.** This directory contains user-level executable programs in Linux system.

   Python libraries and modules are typically stored in directories such as **/usr/lib/python3.x,** which contain the standard library and installed packages. These directories form the core runtime environment for Python programs.

3. <u>User and Permission:</u>

   Python does not run under a dedicated system user or group. Instead, it executes under the context of the user who runs the command. This was verified using process inspection commands, which showed that Python processes are associated with the current logged-in user.

   This design provides flexibility, as any user can execute Python scripts without requiring elevated privileges. However, it also means that access control and security depend on the permissions of the user executing the script.

4. <u>Service Management</u>

   Unlike many other software systems, Python is not managed as a system service. Attempts to start it using service management tools such as systemctl fail because no default service unit exists for Python. This is because Python is an interpreter, not a continuously running background process.

Python programs are executed on demand and terminate once execution is complete. However, Python scripts can be configured to run as services if required, but this involves additional setup beyond the default installation.

```
suyas@LAPTOP-4AKMHPB5:~$ systemctl start python
Failed to start python.service: Interactive authentication required.
See system logs and 'systemctl status python.service' for details.
```

5. Update Model

Python is updated through the system's package manager. When systems updates are performed using command like **sudo apt upgrade,** any available updates for Python are installed automatically.

These updates are maintained by the Linux distribution's maintainers in collaboration with the open-source community. This ensures that security patches and performance improvements are distributed efficiently without requiring manual intervention from the user.

```
suyas@LAPTOP-4AKMHPB5:~$ sudo apt upgrade
[sudo] password for suyas:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  binutils binutils-common binutils-x86-64-linux-gnu cloud-init coreutils
  libbinutils libctf-nobfd0 libctf0 libgprofng0 libsframe1
  python3-software-properties software-properties-common systemd-hwe-hwdb
13 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 6442 kB of archives.
After this operation, 20.5 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 coreutils am
d64 9.4-3ubuntu6.2 [1412 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-hwe-
hwdb all 255.1.7 [3716 B]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libgprofng0
amd64 2.42-4ubuntu2.10 [849 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libctf0 amd6
4 2.42-4ubuntu2.10 [94.5 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libctf-nobfd
0 amd64 2.42-4ubuntu2.10 [98.0 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 binutils-x86
-64-linux-gnu amd64 2.42-4ubuntu2.10 [2463 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libbinutils
amd64 2.42-4ubuntu2.10 [577 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 binutils amd
64 2.42-4ubuntu2.10 [18.2 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 binutils-com
mon amd64 2.42-4ubuntu2.10 [240 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libsframe1
amd64 2.42-4ubuntu2.10 [15.7 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 software-pr
operties-common all 0.99.49.4 [14.4 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-sof
tware-properties all 0.99.49.4 [30.0 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 cloud-init
all 25.3-0ubuntu1~24.04.1 [628 kB]
Fetched 6442 kB in 11s (575 kB/s)
Preconfiguring packages ...
(Reading database ... 40805 files and directories currently installed.)
```

Shell Script Tasks

Script1:

This script generates a system identity report that displays key information about the Linux environment. It uses variables and commands substitution to retrieve system details such as the kernel version (uname -r), current user (whoami), uptime (uptime -p), and distribution name (lsb_release). The script also displays the current date and the user's home directory. Output formatting is handles using multiple echo statements to create a structures and readable layout. This script demonstrates the use of basic shell scripting concepts including variables, command substitution, and formatted output.

```
suyas@LAPTOP-4AKMHPB5:~$ nano script1.sh
suyas@LAPTOP-4AKMHPB5:~$ chmod +x script1.sh
suyas@LAPTOP-4AKMHPB5:~$ ./script1.sh
========================================
   Open Source Audit - Suyash
========================================
Software Chosen : Python
----------------------------------------
OS Distro       : Ubuntu 24.04.4 LTS
Kernel Version  : 6.6.87.2-microsoft-standard-WSL2
Current User    : suyas
Home Directory  : /home/suyas
System Uptime   : up 32 minutes
Current Date    : Thu Mar 26 13:45:52 UTC 2026
----------------------------------------
License Info    : Linux is typically licensed under GPL (General Public Lice
nse)
========================================
suyas@LAPTOP-4AKMHPB5:~$
```

Script 2:

This script checks whether a specified package is installed on the system using the dpkg package manager. If the package is found, it retrieves relevant details such as version and description using dpkg -s. The script uses an if-else condition to handle both installed and non-installed cases. Additionally, a case statement is used to display a short philosophy note about the package, demonstrating how conditional branching can be used to associate meaning with different software tools. This script highlights practical use of package management commands, conditional logic, and pattern matching in shell scripting.

```
suyas@LAPTOP-4AKMHPB5:~$ nano script2.sh
suyas@LAPTOP-4AKMHPB5:~$ chmod +x script2.sh
suyas@LAPTOP-4AKMHPB5:~$ ./script2.sh
========================================
   FOSS Package Inspector
========================================
python3 is installed.
----------------------------------------
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 3.12.3-0ubuntu2.1
Description: interactive high-level object-oriented language (default python
3 version)
Original-Maintainer: Matthias Klose <doko@debian.org>
----------------------------------------
Python: a language designed for readability and community-driven innovation.
========================================
suyas@LAPTOP-4AKMHPB5:~$ |
```

Script 3:

This script audits important system directories by iterating through a predefined list using a for loop. For each directory, it checks whether it exists and then retrieves its permissions, owner, and group information using the ls -ld command combined with awk. Disk usage is calculated using the du -sh command, with errors suppressed for restricted directories. The script also includes a specific check for Python's configuration directory (/etc/python3), demonstrating how conditional logic can be applied verify the presence of software-related files. This script highlights the use of loops, conditional statements, and text processing utilities in shell scripting.

```
----------------------------------------
suyas@LAPTOP-4AKMHPB5:~$ nano script3.sh
suyas@LAPTOP-4AKMHPB5:~$ chmod +x script3.sh
suyas@LAPTOP-4AKMHPB5:~$ ./script3.sh
========================================
      Directory Audit Report
========================================
/etc
  Permissions : drwxr-xr-x root root
  Size        : 4.3M
----------------------------------------
/var/log
  Permissions : drwxr-xr-x root syslog
  Size        : 135M
----------------------------------------
/home
  Permissions : drwxr-xr-x root root
  Size        : 52K
----------------------------------------
/usr/bin
  Permissions : drwxr-xr-x root root
  Size        : 95M
----------------------------------------
/tmp
  Permissions : drwxrwxrwt root root
  Size        : 24K
----------------------------------------

Checking Python configuration directory...
----------------------------------------
/etc/python3 exists
Permissions : drwxr-xr-x root root
========================================
suyas@LAPTOP-4AKMHPB5:~$
```

Script 4:

This script analyzes a log file by reading it line by line using a while loop. It takes the file path and a keyword as command-line arguments, with a default keyword set to “error.” For each line, the script checks if the keyword is present using a case-insensitive comparison and increments a counter accordingly. It also includes error handling to ensure the file exists and is not empty. Finally, the script displays the total number of matches and prints the last five lines containing the keyword using a combination of grep and tail. This demonstrates file handling, conditional logic, and text processing in shell scripting.

```
suyas@LAPTOP-4AKMHPB5:~$ nano script4.sh
suyas@LAPTOP-4AKMHPB5:~$ chmod +x script4.sh
suyas@LAPTOP-4AKMHPB5:~$ ./script4.sh
========================================
        Log File Analyzer
========================================
Error: File  not found.
suyas@LAPTOP-4AKMHPB5:~$ ./script4.sh /var/log/syslog error
========================================
        Log File Analyzer
========================================

Keyword 'error' found 21 times in /var/log/syslog
----------------------------------------
Last 5 matching lines:
2026-03-19T16:48:26.502761+00:00 LAPTOP-4AKMHPB5 kernel: WSL (217) ERROR: Ch
eckConnection: getaddrinfo() failed: -5
2026-03-26T13:14:35.142468+00:00 LAPTOP-4AKMHPB5 systemd[1]: apport-autorepo
rt.path - Process error reports when automatic reporting is enabled (file wa
tch) was skipped because of an unmet condition check (ConditionPathExists=/v
ar/lib/apport/autoreport).
2026-03-26T13:14:35.142474+00:00 LAPTOP-4AKMHPB5 systemd[1]: apport-autorepo
rt.timer - Process error reports when automatic reporting is enabled (timer
based) was skipped because of an unmet condition check (ConditionPathExists=
/var/lib/apport/autoreport).
2026-03-26T13:14:35.144855+00:00 LAPTOP-4AKMHPB5 kernel: RAS: Correctable Er
rors collector initialized.
2026-03-26T13:15:02.242221+00:00 LAPTOP-4AKMHPB5 kernel: WSL (218) ERROR: Ch
eckConnection: getaddrinfo() failed: -5
========================================
suyas@LAPTOP-4AKMHPB5:~$
```

Script 5:

This script generates a personalized open-source manifesto by taking user input through interactive prompts. It collects responses using the read command and stores them in variables. These inputs are then combined into a structured paragraph using string concatenation and written to a text file using output redirection (>>). The script also includes the current date and dynamically generates the output filename based on the user. Finally, the generated manifesto is displayed using the cat command. This script demonstrates user interaction, file handling, and dynamic content generation in shell scripting.

```
suyas@LAPTOP-4AKMHPB5:~$ nano script5.sh
suyas@LAPTOP-4AKMHPB5:~$ chmod +x script5.sh
suyas@LAPTOP-4AKMHPB5:~$ ./script5.sh
========================================
   Open Source Manifesto Generator
========================================

1. Name one open-source tool you use every day: Python
2. In one word, what does 'freedom' mean to you? The ability to do anything
without much restrictions.
3. Name one thing you would build and share freely: Projects related to real
-world problems.

Manifesto saved to manifesto_suyas.txt
----------------------------------------

Open Source Manifesto - 26 March 2026
----------------------------------------

I use tools like Python in my daily work, which represent the power of open
collaboration.
To me, freedom means The ability to do anythiing without much restrictions.,
 and it is this principle that drives the open-source movement.
In the future, I aim to build and share Projects related to real-world probl
ems. so that others can learn, improve, and innovate upon it.
I believe that knowledge should not be restricted, but shared openly to crea
te a stronger and more inclusive technological ecosystem.
========================================
suyas@LAPTOP-4AKMHPB5:~$
```

PART C – THE FOSS ECOSYSTEM

Python does not operate as a standalone tool but as part of a large and interconnected open-source ecosystem. At its core, Python depends on lower-level technologies such as the C programming language, as the Python interpreter itself is implemented in C. It also relies on system-level components provided by Unix-like operating systems, including file handling, memory management, and process control. Compilers like GCC are used during development and distribution, further linking Python to other foundational open-source tools.

At the same time, Python has enabled the creation of a vast number of tools and frameworks built on top of it. Popular web frameworks such as Django and Flask allow developers to build scalable web applications, while libraries like NumPy, Pandas, and TensorFlow have made Python a dominant language in data science and artificial intelligence. This layered structure shows how python acts as a platform that supports innovation across multiple domains.

In the context of web technologies, Python often replaces or complements traditional components of the LAMP stack. Instead of PHP, developers may use python frameworks to build dynamic web applications, while still relying on Linux, Apache and MySQL in the backend. This demonstrates python's flexibility and its ability to integrate seamlessly into existing open-source architectures.

The python ecosystem is supported by a strong global community governed by the Python Software Foundation (PSF). Development is coordinated through open platforms such as Github, where contributors from around the world collaborate on improving the language. The ecosystem also includes PyPI (Python Package Index), which hosts thousands of open-source libraries that can be easily installed and used. Community interaction takes

place through forums, mailing lists, and conferences, ensuring continuous growth and improvement of the language.
Overall, Python's role in the open-source ecosystem is both foundational and expensive. It depends on earlier technologies while simultaneously enabling new ones, creating a cycle of continuous innovation driven by collaboration and shared knowledge.

PART D – OPEN-SOURCE VS PROPRIETARY (MATLAB)

Dimension	Open-Source	Matlab
Cost	Free to use, modify and distribute,	Requires paid license, often expensive
Security	Source code is open for inspection, allowing community auditing.	Source code is closed, users must trust the company.
Support and reliability	Community support + extensive documentation	Official support from company (MathWorks)
Freedom to modify	Full freedom to modify and redistribute	No access to source code, modifications restricted.
Community vs corporate control	Community-driven development	Controlled by a single organization.
Overall Verdict	Highly flexible with vast libraries	Strong in numerical computing and specialized engineering tools

CONCLUSION

Python and MATLAB both serve important roles, but they differ significantly in philosophy and usage. Python offers flexibility, cost-effectiveness, and a vast ecosystem, making it suitable for a wide range of applications from web development to artificial intelligence. Its open-source nature allows continuous improvement and transparency. On the other hand, MATLAB provides a more

controlled environment with specialized tools for numerical computing and engineering tasks, along with dedicated customer support. This makes it more suitable for organizations that require stability and official backing.

Based on this comparison, Python is generally more suitable for real-world deployment due to its flexibility, scalability, and strong community support. It allows developers to experiment, innovate and build without restrictions. However, MATLAB may still be preferred in niche domains where its specialized toolboxes provide a significant advantage. Personally, I would choose python for most applications and would also consider contributing to its ecosystem, as its open-source nature encourages collaboration and continuous improvement.